

**RELATÓRIO FINAL
DE INICIAÇÃO CIENTÍFICA**

Formatação Base: 1) Redação em língua portuguesa, 2) espaçamento entre linhas de 1,5 cm, 3) sem espaços entre parágrafos (zero antes e depois), 4) fontes Times New Roman ou Arial, 5) tamanho 12, mesmo nos gráficos e tabelas. 6) páginas numeradas, com margem esquerda e superior de 3,0 cm e direita e inferior de 2,0 cm.

Dados do Projeto de IC

Título do projeto: Reconhecimento de Placa de Carro com Aplicação em Controle de Acesso Estacionamento.

Área de Conhecimento: Ciências Exatas e da Terra

Subárea de Conhecimento: (☒) Ciência da Computação (☐) Não se aplica

Dados do Proponente

Nome Completo: Pedriana de Jesus Pavão Castro

Titulação: (☐) Doutor(a) (☒) Mestre(a)

E-mail: Pedriana de Jesus Pavão Castro

Telefone: (98) 98402-3579

Link Currículo Lattes: <https://lattes.cnpq.br/2879671600184085>

Dados do(a) Aluno(a) de Iniciação Científica

Nome Completo do(a) Aluno(a): Samuel Pereira da Silva Varão

Modalidade de Iniciação Científica: (☒) Bolsista (☐) Voluntário(a)

Curso: Sistemas de Informação

E-mail: samuelpereiradasilvavarao@gmail.com

Telefone: (61) 99974-4607

Link Currículo Lattes: <https://lattes.cnpq.br/4265937860587897>

Título do Plano de Trabalho: Desenvolvimento e Implementação de Sistema Automatizado de Reconhecimento Veicular.

RESUMO

Introdução: Na sociedade atual os carros tornaram-se indispensáveis para a locomoção urbana. Com um número de veículos crescente percebe-se a necessidade de soluções inteligentes para as adversidades deste aumento. **Objetivo:** Através de um algoritmo de ALPR (*Automatic License Plate Recognition*) trazer eficiência a processos de acesso e saída em locais públicos ou privados, os tornando mais eficientes e também possibilitando uma maior monitoração e por consequência segurança aos locais de aplicação. **Material e método:** O processo de construção de uma ALPR parte em síntese de três processos principais. O treinamento de uma Rede Neural Convolucional para especializar-lá na detecção de placas; O processamento da imagem da placa detectada e a extração dos caracteres através de uma OCR (*Optical Character Recognition*). **Resultados:** O algoritmo proposto demonstrou uma boa capacidade de detectar placas de carro Brasileiras do modelo Mercosul, porém a OCR manifestou dificuldades de reconhecimento comuns em caracteres específicos presentes nas placas.

Palavras-chave: Visão Computacional, Placas, Mercosul, YOLO, Tesseract.

ABSTRACT

In modern society, cars have become indispensable for urban mobility. With the growing number of vehicles, the need for intelligent solutions to address the challenges of this increase becomes evident. Through an ALPR (*Automatic License Plate Recognition*) algorithm, it is possible to bring efficiency to access and exit processes in public or private locations, making them more efficient and also enabling greater monitoring and, consequently, increased security in the places where it is applied. The process of building an ALPR system essentially involves three main stages: training a Convolutional Neural Network to specialize in license plate detection; processing the detected license plate image; and extracting the characters through OCR (*Optical Character Recognition*). The proposed algorithm demonstrated a good ability to detect Brazilian Mercosur-style license plates; however, the OCR showed common difficulties with specific characters present on the plates.

Key-words: Computer vision, License plate, Mercosul, YOLO, Tesseract.

INTRODUÇÃO / JUSTIFICATIVA

O deslocamento nas cidades influencia o cotidiano de variadas maneiras, abrangendo trajetos para o emprego, instituições de ensino ou residências. A utilização de automóveis consolidou-se como a modalidade de transporte mais usual e prática nas metrópoles. Todavia, conforme dados do Instituto Brasileiro de Geografia e Estatística (IBGE), a frota veicular nacional expandiu-se de aproximadamente 42 milhões, em 2012, para uma marca superior a 60, em 2022.

O crescimento volumétrico acarreta problemáticas relativas ao ordenamento e à fluidez do tráfego nos corredores principais durante os períodos de pico, resultando em congestionamentos e suas respectivas consequências (RESENDE; SOUZA, 2009). Nesse sentido, a aplicação de Sistemas Inteligentes de Transporte (ITS) apresenta-se como a via primordial para as transformações pretendidas, assegurando mecanismos de fiscalização e proteção tanto na esfera pública quanto na particular (FIGUEIREDO, 2005).

Consequentemente a entrada em estacionamentos correlaciona-se ao fluxo de entrada e saída de carros das vias externas até este espaço, integrando a competência de administração das vagas, onde gargalos em horários de alta demanda provocam perturbações acentuadas. De acordo com Lopes et al. (2020), “a automação do controle de acesso por meio de reconhecimento de placas representa uma solução de baixo custo e alto impacto, com aplicabilidade direta na segurança urbana e institucional”. Integrada aos ITS, encontra-se a tecnologia de Reconhecimento Automático de Placas de Veículos (ALPR – *Automatic License Plate Recognition*) que, ao utilizar métodos de aprendizado de máquina, constitui a estratégia para atingir níveis satisfatórios de mobilidade e proteção no ambiente institucional, promovendo avanços operacionais e organizacionais no cotidiano universitário.

Por conseguinte, a elaboração de um software capaz de identificar placas automotivas torna-se imprescindível no contexto institucional, colaborando não apenas com o aparato de segurança, mas com o incremento significativo da qualidade operacional diária, ao conferir celeridade e precisão nos processos de entrada e saída (CONDOLETTE, 2023). Dessa forma, propicia-se o aperfeiçoamento das dinâmicas estruturais e funcionais da rotina acadêmica.

MATERIAL E MÉTODOS

Os elementos exibidos integram uma área da inteligência artificial denominada *Visão Computacional*, o qual aborda: “desafios em igualar a capacidade humana de interpretar o mundo visualmente, devido à necessidade de modelos complexos para compreender formas, texturas e emoções em imagens” (SZELISKI, 2022).

O fluxo operacional do sistema destinado à identificação de placas veiculares proposto segmenta-se em três fases principais, as quais comportam etapas secundárias internas.

O ciclo é composto por: **Deteccção, Processamento de Imagem e Reconhecimento de Caracteres**. Nesse escopo, a detecção de imagens é obtida após o treinamento de uma CNN (Convolutional Neural Network). O modelo utilizado é o YOLOv8 que treinará com dataset de imagens de placas de carro. Uma vez que a imagem da placa é detectada, o tratamento será efetuado através dos módulos de processamento de imagem da biblioteca OpenCV disponibilizada pela Google. Já a extração textual dos caracteres é realizada pela ferramenta Tesseract (OLIVEIRA; SAMPAIO, 2024).

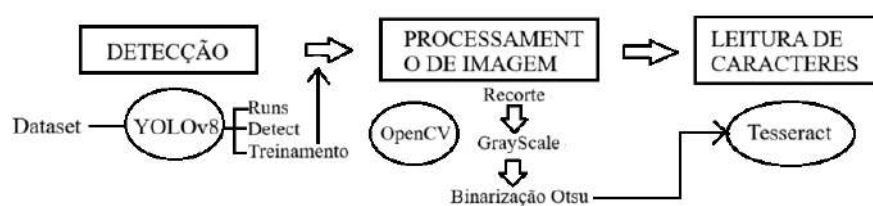


Figura 1 – *Pipeline* da Metodologia.

Fonte: Autor.

1. DETECÇÃO E TREINAMENTO COM O YOLO

O (You Only Look Once) YOLO, é um modelo de treinamento desenvolvido pela Ultralytics, com capacidade de segmentar, classificar e detectar a partir de imagens e vídeos os objetos desejados, oportunizando o treinamento personalizado ou o uso de versões pré-treinadas. A versão utilizado neste projeto será o YOLOv8, por sua alta precisão,

velocidade e leveza na inferência (YOLO, 2023). Como cada parte do programa ficará responsável por uma parte do processo. O YOLO desempenha o papel de fazer somente a detecção da placa.

O procedimento tem início um conjunto de dados (*Dataset*) composto por fotografias de placas de veículos destinadas à capacitação do YOLOv8. Tais imagens contêm regiões delimitadas no objeto de interesse que orientam o aprendizado da nossa rede neural convolucional (*Labels*).

A Rede Neural Convolucional ou CNN (*Convolutional Neural Network*) opera de maneira análoga ao cérebro humano, contudo, é estruturada por uma sucessão de funções matemáticas que viabilizem o aperfeiçoamento a cada iteração. Conforme descreve Matarozzi (2022, p. 12), estas arquiteturas “levam esse nome a partir da operação de convolução (ou melhor, relação cruzada) que é realizada em uma camada da rede”.

Dessa maneira, a cada ciclo de treinamento, o modelo torna-se mais apurado. Todavia, para que a acurácia do sistema de identificação atinja níveis elevados, ele deve dispor de um banco de dados que seja, simultaneamente, robusto e dotado de diversidade, permitindo a simulação de uma ampla gama de cenários e variáveis (NETO, 2023).

1.1. O Dataset

O Dataset escolhido possui 442 imagens de diversas placas de carros em variados ambientes e estados físicos. Tanto no padrão antigo quanto no padrão moderno brasileiros. Algumas placas apresentam-se com diferentes tipos de avarias em sua composição, como cortes ou paleta de cores discrepantes, além de diversos ângulos não convencionais. O dataset escolhido também já possui as *labels* (Marcações) localizadas adequadamente nas áreas onde localizam-se as placas, essas marcações estão adequadamente dispostas pelas imagens de forma propícia durante a execução do aprendizado.

Para que o modelo consiga seguir um caminho consistente durante o treinamento, assim possibilitando que possa obter a capacidade de identificar as placas de carro, delimita-se então um escopo conciso, neste projeto, especificamente, foi então escolhido como objetivo a identificação do modelo de placas de carro brasileiro Mercosul. Essa definição clara facilita o treinamento e torna o processo de convolução mais eficiente e rápido (ROBOFLOW, 2025). Sendo assim, o fato dele possuir então somente uma classe,

“Placas-dec-3-classes-mercosul-a440m” o treinamento ocorrerá de forma muito mais rápida e preciso.

```
train: train/images
val: train/images

nc: 1
names: ['Placa_mercosul_carro']

roboflow:
  workspace: samuels-workspace-8un26
  project: placas-dec-3-classes-mercosul-a440m
  version: 1
  license: CC BY 4.0
  url: https://universe.roboflow.com/samuels-workspace-8un26/placas-dec-3-classes-mercosul-a440m/dataset/1
```

Figura 2 – *data.yaml* do Dataset.

Fonte: Autor

1.2. Treinamento da Rede Neural Convolutacional.

O processo da Rede Neural Convolutacional é constituído pelo refinamento constante conforme o processamento dos pesos das imagens disponíveis para o treinamento com base no dataset escolhido, desenvolvendo-se então durante a execução do treinamento. Deste modo, tal operação de percorrer todas as imagens irá gerar novos dados, e são esses dados que servirão de métrica para as novas comparações das convoluções conseguintes na execução do treinamento.

O processo neste algoritmo foi configurado especificamente para cinquenta *Epochs* (Épocas). São essas épocas onde os cálculos do processo de convolução acontecem. O YOLO nos modelos YOLOv5/v8 sendo um modelo do tipo EMA (*exponential Moving Average*) é uma arquitetura onde são criadas cópias. **A primeira:** Responsável pela inferência com os gradientes e os cálculos dos pesos. **A segunda:** Recebe os resultados obtidos na anterior e começa a tratá-los de maneira suave. Assim cria-se uma curva de aprendizagem gradual, diminuindo ruídos (YOLO, 2023). Durante o processo, o YOLO irá gerar uma série de métricas importantes como: precisão, *recall* e matriz de confusão. Que possibilitaram um diagnóstico da eficiência e acurácia do modelo.

```
def main():  
    print("Iniciando o Treinamento do YOLOv8...")  
  
    model = YOLO("yolov8n.pt")  
  
    #treinamento com os dados  
    resultados = model.train(  
        data="dataset/data.yaml", # Caminho do arquivo de configuração  
        epochs=50,                # vezes que ele vai ver o dataset inteiro  
        imgsz=640,                # Tamanho de redimensionadas de imagens para treino  
        batch=8,                  # Quantidade de imagens processadas por vez pela GPU  
        device=0,                 # força o uso da Placa de Vídeo NVIDIA  
        plots=True,               # Gera gráficos  
        project="Treinamento",   # Nome da pasta dos resultados  
        name="Placas_Mercosul"   # Nome da subpasta dos resultados  
    )
```

Figura 3 – *Script* de treinamento comentado.

Fonte: Autor

2. PROCESSAMENTO DE IMAGEM: OPENCV

O OpenCV é uma biblioteca multiplataforma de código aberto com foco na visão computacional. Essa biblioteca possui muitas funcionalidades, dentre elas a edição e manipulação de imagens. (OPENCV, 2025). Por possuir diversos módulos de processamento, dentre as diversas opções, foram selecionados apenas três os filtros morfológicos. Entretanto no projeto foram aplicados somente dois, também há comandos de *Smart Crop* para fazer o processo de limpeza das bordas e recorte. Os Filtros morfológicos servirão para facilitar que a OCR faça a leitura dos caracteres da placa, deste modo garantindo um resultado mais preciso.

O processamento de imagem é então sequencializado nas seguintes etapas:

- I. **SMART CROP:** O YOLO detecta a placa de carro do modelo Mercosul, logo envia a imagem para o OpenCV que recorta a imagem no Retângulo de detecção.

```
#SMART CROP  
altura = y2 - y1  
largura = x2 - x1  
  
corte_topo = int(altura * 0.25) # Tira 25% de cima (Faixa Azul)  
corte_lateral = int(largura * 0.05) # Tira 5% de cada lado (Bordas Pretas)
```

Tabela 1 – *Smart Crop*

Fonte: Autor

- II. **GRAYSCALE:** A imagem recortada é então convertida para *Grayscale* (escala de cinza). Isso permitirá que o próximo processo ocorra sem que aconteçam avarias ou discrepâncias de elementos não desejados na imagem. (Tabela 2)
- III. **BINARIZAÇÃO:** Utilizando o método Otsu, a segmentação do objeto de destaque da imagem de seu fundo, com o uso de duas funções gaussianas, assumindo um histograma bimodal na imagem (BORGES, 2013). Assim, a fazendo ficar em preto e branco, dando contraste evidente nos elementos a serem lidos, facilitando consideravelmente leitura da OCR, evitando que borrões ou manchas que não sejam os caracteres da placa sejam lidos.



Figuras 4 – Placa recortada da imagem e depois binarizada com a técnica Otsu

```
if placa_recortada.size != 0:  
    cinza = cv2.cvtColor(placa_recortada, cv2.COLOR_BGR2GRAY)  
    _, binarizada = cv2.threshold(cinza, 0, 255, cv2.THRESH_BINARY cv2.THRESH_OTSU)
```

Tabela 2 – *GrayScale* e binarização Otsu.

Fonte: Autor

3. LEITURA DE CARACTERES: TESSERACT E O PÓS-PROCESSAMENTO

3.1. Tesseract

O Tesseract é um OCR (Optical Character Recognition) um software dedicado ao reconhecimento de caracteres. Atualmente é mantido pelo Google e foi originalmente criado pela HP. (NETO, 2023). O tesseract então sendo uma grande biblioteca que possui diversos caracteres conhecidos receberá a imagem binarizada e irá comparar com padrões que o próprio Tesseract possui e conhece, então retornará uma string com os caracteres identificados

após a leitura.

Apesar dos processos de binarização colaborarem para uma maior precisão da leitura. O Tesseract também pode ser treinado em uma fonte específica, facilitando ainda mais o processo de identificação (LEITE, ANTONELLO, 2017).

3.2. Pós-processamento

O pós-processamento é uma etapa importante pois apesar da OCR ser capaz de identificar os caracteres da imagem tratada, por vezes, acaba cometendo trocas entre caracteres relativamente similares, seja pelo formato da letra ou número, ou por vezes pela presença pontos de ruído, manchas ou borrões na imagem que são grandes o suficiente para serem processadas nos passos do tratamento da imagem com os filtros morfológicos do OpenCV, acarretando então na distorção da forma como o Tesseract interpreta e faz a leitura. Alguns dos erros mais comuns cometidos são as trocas dos seguintes caracteres:

1 → I

4 → A

8 → B

O → 0

Por isso, para minimizar a quantidade de erros, podemos utilizar *Regex* (expressões regulares). As expressões regulares servirão como uma espécie de peneira. Após a leitura e a geração de uma série de strings, essa mesma série pode passar por uma validação antes de ser extraída. As placas de trânsito brasileiras podem ser consideradas uma expressão regular. Ela possui uma quantidade limitada de elementos, no total sete, sendo eles números e letras que seguem um padrão lógico padronizado e único:

Letra - Letra - Letra - Número - Letra - Número - Número.



Figura 5 – Placa do Mercosul e seu padrão.

Fonte: Confederação Nacional do Transporte.

Deste modo, ao gerar uma string onde por exemplo, foi identificado um caractere a mais ou a menos fazendo com que o resultado lido possua mais caracteres ou menos caracteres na expressão regular delimitada, fazendo a utilização de uma série de comandos lógicos, podemos já filtrar as detecções que não possuem a quantidade correta de caracteres da placa.

O regex então, da mesma maneira em um outro cenário onde na terceira posição foi identificado um número “1”, graças ao regex, esse resultado então seria considerado inválido e descartado, pois a expressão regular sempre dirá que a terceira posição não pode ser um número, mas sim uma letra. Entretanto, ao invés de descartar, podemos adicionar uma camada de correção. O resultado então será aceito e através do uso de um dicionário poderá ser corrigido. Como a placa possui uma quantidade fixa de caracteres e já que a terceira posição em uma placa brasileira no modelo Mercosul sempre será uma letra, podemos elaborar um dicionário contendo as trocas mais frequentes na OCR. Por tanto com o Regex e o dicionário, o “1” identificado é prontamente corrigido para uma letra, o “I”. Retornando assim uma string mais aproximada da placa lida.

```
def corrigir_placa_mercosul(texto_lido):
    texto = ''.join(e for e in texto_lido if e.isalnum()).upper()

    if len(texto) < 7:
        return None

    if len(texto) > 7:
        texto = texto[-7:]

    #Dicionários de correção para as confusões mais comuns do Tesseract
    letras_erradas = {'0': '0', '1': '1', '2': '2', '3': '3', '4': '4', '5': '5', '6': '6', '7': '7', '8': '8', '9': '9'}
    numeros_errados = {'0': '0', '1': '1', '2': '2', '3': '3', '4': '4', '5': '5', '6': '6', '7': '7', '8': '8', '9': '9'}

    placa_corrigida = ""

    for i, char in enumerate(texto):
        if i in [0, 1, 2, 4]:
            placa_corrigida += letras_erradas.get(char, char)
        else:
            placa_corrigida += numeros_errados.get(char, char)

    if re.match(r'^[A-Z]{3}[0-9][A-Z][0-9]{2}$', placa_corrigida):
        return placa_corrigida
    else:
        return None
```

Figura 6 – Pós-processamento; função para correção e filtragem usando *Regex* e dicionário.

Fonte: Autor.

3.2.1 O Warp Perspective.

O Tesseract por ter sido produzido com objetivo de ser utilizado em documentações regulares e consistentes, possui uma série de limitações. Em cenários onde a inclinação dos caracteres é maior, a OCR comete erros mais acentuados, já que sua construção é feita para leitura e identificação de textos horizontais e alinhados. Para a resolução desse problema, propõe-se o uso de filtros de correção de perspectiva, onde se ajusta a imagem de maneira que torne a leitura da OCR mais eficaz.

RESULTADOS E DISCUSSÃO

Os tópicos a seguir tem como objetivo evidenciar cada uma das saídas e resultados obtidos de cada um dos procedimentos realizados pelo programa. Para tornar cada uma desses produtos obtidos evidenciados em suas categorias, assim sendo expostos de forma mais clara e objetiva, essa parte foi dividida em cinco seções principais, sendo elas: **Processamento da imagem; uso dos filtros morfológicos; Identificações feitas pela OCR; Saídas de imagem e vídeo; Treinamento do YOLO e suas métricas; Avaliação do sistema do sistema; Limitações do projeto e Propostas de aperfeiçoamento.**

1. Processamento de Imagem; Uso dos filtros morfológicos

O projeto foi construído de maneira que possua a capacidade de identificar placas tanto em imagens estáticas quanto em vídeos, pré-gravados ou em tempo real. Os algoritmos que fazem o processamento tanto de um quanto do outro possuem similaridades com variações particulares no código, pois como um vídeo é um conjunto de diversas imagens *frames*, seu tratamento ocorre em um procedimento contínuo através de uma função recursiva.

Sendo assim, as saídas de ambos os algoritmos seguem a mesma lógica:

Imagem detectada → Recorte da área de detecção → Aplicação de filtros → Leitura da OCR

No teste de A, a placa é identificada imediatamente. A execução do processamento de imagem aconteceu satisfatoriamente. Vale ressaltar o bom estado de captura da imagem. A placa não possui avarias evidentes, angulações estranhas, iluminação que dificulte o processo de identificação ou quaisquer outros problemas de distorções. Proporcionando como observado na imagem (b), a imediata leitura dos caracteres.

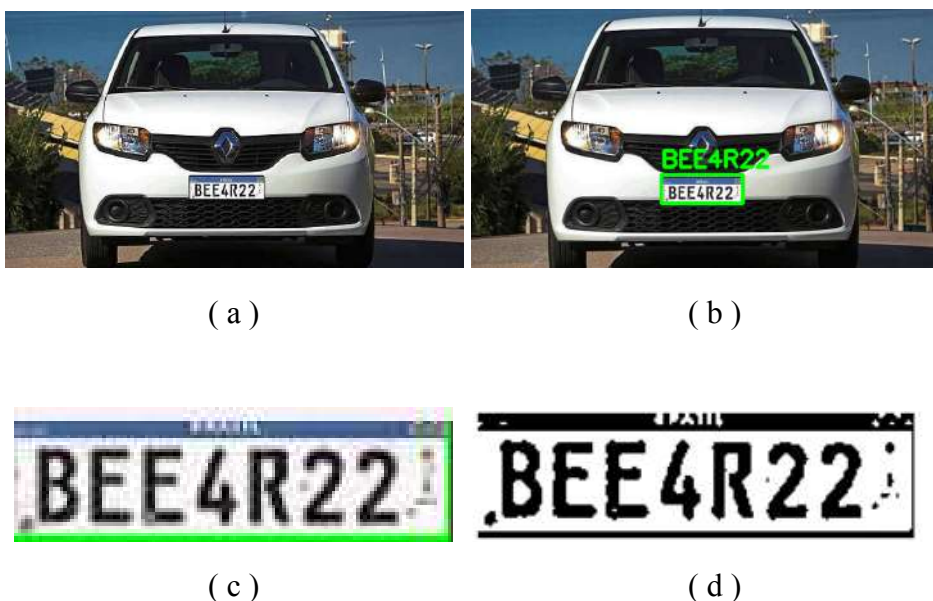


Figura 7 – Teste A. (a) Imagem Original (Fonte: Portal6). (b) detectada pelo YOLO.
(c) Imagem recortada na área detectada. (b) Imagem binarizada.

Na etapa do processamento de imagens, especificamente após a binarização da imagem, o seu tamanho é dobrado, essa ampliação servirá para garantir uma melhor extração e leitura da OCR. Evidenciando as regiões e espaços entre os caracteres e valorizando suas características e contornos, para assim evitar cenários que podem confundir a OCR.

No Teste B, a placa foi detectada com sucesso, o processo seguiu normalmente com a imagem sendo recortada na área detectada. Importante ressaltar que a aplicação do filtro morfológico responsável converte a imagem para tons de cinza antes da binarização existe apesar deste processo ser omitido nas Figuras 5 e 6. Uma avaliação significativa é que apesar do bom estado da imagem, em sentido de iluminação e qualidade de captura, falta de avarias que dificultariam a leitura, graças a angulação da placa, os caracteres não foram detectados

imediatamente na imagem.

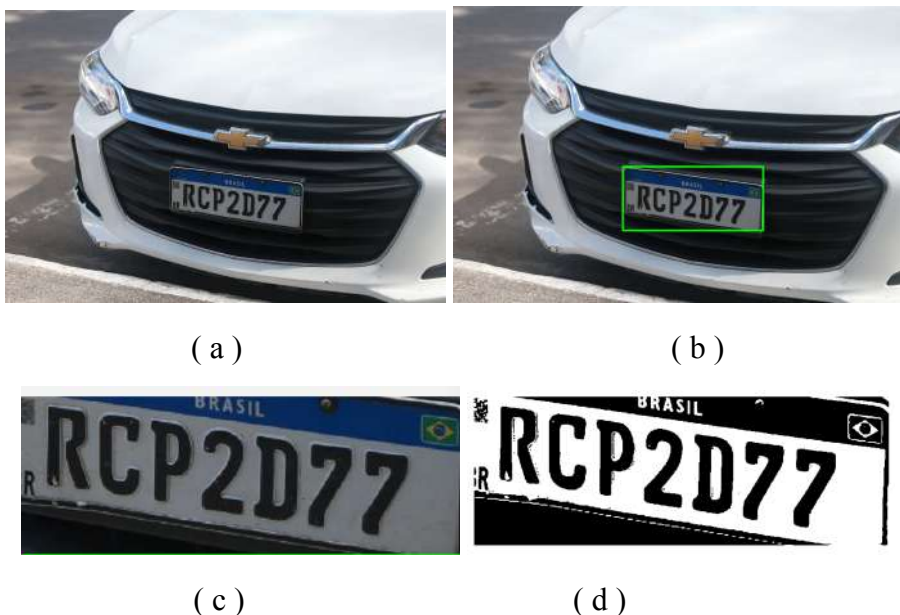


Figura 8 – Teste B (a) Imagem original (fonte: Estadão). (b) detectada pelo YOLO.
(c) Imagem recortada na área detectada. (d) Imagem Binarizada.

2. Identificações feitas pela OCR; Saídas de Imagem e Vídeo.

Após o processamento, vem a etapa da OCR que irá fazer a identificação dos caracteres na imagem binarizada. Apesar de o Tesseract ter a capacidade de se especializar em um tipo específico de fonte, como a variação da FE Engschrift utilizada oficialmente pelo Brasil nas placas veiculares brasileiras no modelo do Mercosul. Este projeto utiliza o Pytesseract com suas bibliotecas nativas, treinadas para identificação de diversos tipos de caracteres em diversas fontes.

Há dois tipos de saídas. A saída da leitura e extração da string vinda de uma imagem e a extração da string vinda de uma captura em tempo real. Antes da string ser enviada, ela passa pelas etapas de validação do Regex.

2.1. Imagens Estáticas

A saída do Teste A demonstra como a leitura foi bem sucedida, assim como o

resultado obtido da leitura passou nas etapas de verificação, sendo assim mostrado no terminal.

```
Analizando a imagem...  
0: 384x640 1 Placa_mercosul_carro, 43.6ms  
Speed: 13.2ms preprocess, 43.6ms inference, 11.2ms postprocess per image at shape (1, 3, 384, 640)  
PLACA VALIDADA: BEE4R22  
Pressione qualquer tecla na janela da imagem para fechar...
```

Figura 9 – Saída do teste A: extração da Imagem.

Já a saída do Teste B demonstra outro cenário que o algoritmo produz. O processo de identificação da placa ocorreu com sucesso graças ao treinamento da CNN. Porém a leitura falhou. Existem muitas variáveis prováveis para a falha na captura, dentre elas pode-se dizer que os caracteres não foram identificados graças a inclinação da imagem (Figura 8 – (b)).

```
Analizando a imagem...  
0: 448x640 1 Placa_mercosul_carro, 32.2ms  
Speed: 14.2ms preprocess, 32.2ms inference, 8.4ms postprocess per image at shape (1, 3, 448, 640)  
Placa detectada, mas a leitura falhou:
```

Figura 10 – Saída do teste B: extração da imagem.

Há cenários onde a captura poderia ser efetiva, sem uma saída, pois, como o código aplica etapas de validação, caso um caractere específico em uma posição inexistente como uma 8 ou 9 posição numa expressão regular que consiste em somente 7 posições, não haveria saída. Também, como saídas onde letras são identificadas nas posições numéricas não seriam mostradas no terminal. Para isso serve como apoio de verificação de um dicionário. Através dele o resultado então seria corrigido, e somente após a correção, teria a saída exibida no terminal.

2.2. Captura em Tempo Real

Todos os procedimentos descritos na última sessão ocorrem na captura em tempo real. Como um vídeo é composto de centenas de imagens sequenciais, o processo é o mesmo. Com a diferença que aqui, a função responsável é recursiva.

No teste A, conseguimos ver bem como ocorre o processo de identificação de um

caractere e como o tesseract segue rigidamente o Regex, mas ainda cometendo confusões e trocas durante a execução do programa antes de estabilizar definitivamente.

```
[ Frame 10 ] Placa identificada: BEE6R22
[ Frame 11 ] Placa identificada: BEE4R22
[ Frame 12 ] Placa identificada: BEE5R22
[ Frame 13 ] Placa identificada: BEE4R22
[ Frame 14 ] Placa identificada: BEE4R22
[ Frame 15 ] Placa identificada: BEE4R22
[ Frame 17 ] Placa identificada: BEE4R22
```

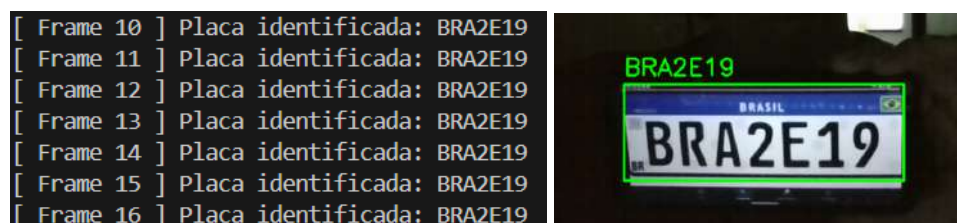
Figura 10 - Saída do teste A: Extração em tempo real via Webcam.

No teste B é perceptível que apesar da leitura da imagem individual ter falhado, a captura em tempo real através da webcam conseguiu extrair informações da placa capturada. É notável também certa dificuldade na identificação de dois caracteres, causando certa inconsistência nessas posições específicas. Porém nas demais posições, a leitura demonstrou-se sólida.

```
[ Frame 12 ] Placa identificada: RCP2077
[ Frame 13 ] Placa identificada: RCP2077
[ Frame 14 ] Placa identificada: RCP2D77
[ Frame 15 ] Placa identificada: RCP2N77
[ Frame 16 ] Placa identificada: RCP2077
[ Frame 17 ] Placa identificada: RCP2077
[ Frame 18 ] Placa identificada: RCP2D77
```

Figura 11 – Saída do teste B: Extração em tempo real via WEBcam.

Quando se é utilizada a imagem de uma placa, sem o carro, em angulação perfeita, a leitura em tempo real se mostra consistente e sólida. Sendo filmada pela Webcam a imagem (figura 5) é utilizada como exemplo na próxima figura, evidenciando a influência de condições favoráveis assegura a assertividade do programa.



(a)

(b)

Figura 12 – Extração em tempo real Webcam (Figura 5),

(a) Saída terminal. (b) Print da Webcam

3. Treinamento do YOLO e Suas Métricas

Para compreender os dados obtidos durante o treinamento da rede neural convolucional, é importante elucidar os termos utilizados nas métricas de treinamento e cálculo de cada um. Os mais importantes são a Precisão, *Recall* e o *F1-Score*.

3.1 Métricas importantes utilizadas

I. Precisão

A precisão consistirá na quantidade de inferências feitas pelo modelo durante o seu processo de treinamento, as inferências são então o momento em que o modelo aponta para uma imagem e diz que a imagem apontada possui a classe estabelecida para ser encontrada. Neste caso, placas de carro brasileiras no modelo do Mercosul.

$$\text{Precisão} = \frac{\text{Positivos verdadeiros}}{\text{Positivos verdadeiros} + \text{Falsos positivos}}$$

II. Recall

Quando o modelo está em treinamento, o dataset já possui marcações definidas nos locais onde se localizam a placa. Portanto, o recall funcionaria como uma avaliação perante as inferências do modelo. Ele então analisa a quantidade de objetos da classe desejada na imagem e na quantidade de objetos encontrados pelo modelo.

$$\text{Recall} = \frac{\text{Positivos verdadeiros}}{\text{Positivos verdadeiros} + \text{Falsos Negativos}}$$

III. F1-Score

No treinamento de uma rede neural convolucional, se prioriza que o modelo mantenha o equilíbrio entre as duas métricas anteriores, a métrica utilizada para manter esse equilíbrio é o F1-Score. Ela ajuda a corrigir o modelo se ele tiver muita precisão e pouco recall (ou vice-versa), forçando a busca por equilíbrio.

$$F1 = 2 \times \frac{\text{Precisão} \times \text{Recall}}{\text{Precisão} + \text{Recall}}$$

Esses valores métricos são importantes para entender o comportamento lógico por trás

de cada uma das operações feitas durante o treinamento do modelo, servindo como alicerce fundamental para diversos tipos de ajustes, esses dados fazem com que tenhamos o poder de tornar esses processo mais eficazes e eficientes, possibilitando que aplicação nas ocasiões necessárias, ocorram de forma mais acuradas (SCHLOSSER, et al. 2026).

3.2 Gráficos gerados durante o treinamento e interpretações

O YOLO por padrão gera diversos gráficos durante o treinamento que servirão como diagnóstico daquilo que ocorreu durante as diversas épocas executadas, sendo então os resultados de nosso modelo e de seu desempenho.

A Figura 13, mostra o gráfico da precisão, como vemos nos eixos X e Y, o modelo começa com uma baixa precisão. Conforme os treinamentos vem a exigência de mais confiança de acordo com as inferências e comparações com as labels. Vai ocorrer então que a linha de precisão aumentará assim como a confiança do sistema. Uma vez que o modelo atinge o limite 1.0, se se pode afirmar que a precisão está alta, abaixo disso indica que o modelo disparou demasiadamente falsos positivos, assim diminuindo a confiabilidade de detecção da classe escolhida. A figura 13 demonstra uma curva satisfatória do modelo treinado nesse projeto, garantindo uma baixíssima chance de falsos positivos.

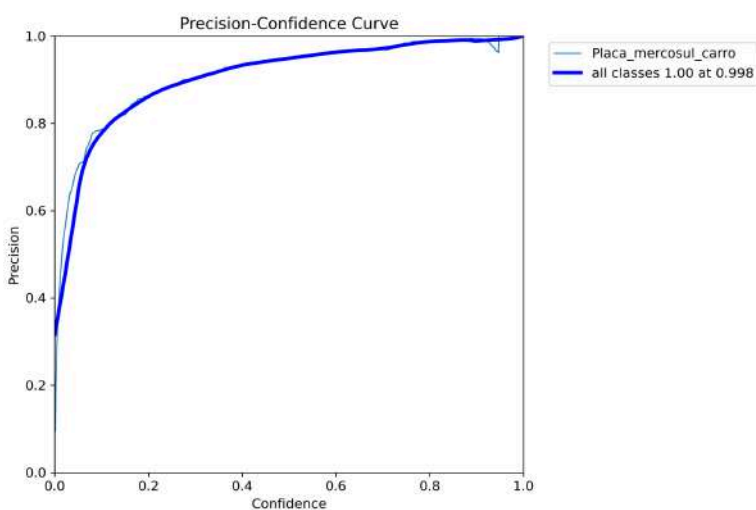


Figura 13 – Precisão: Curva de confiança gerada pelo YOLO

A Figura 14 é o F1-score do modelo. Faz a mescla de dois dados importantes: a precisão e o recall. O topo exato da função deste gráfico aponta para **0.334 no eixo X** e **0.94 no eixo Y**. Isso diz diretamente que a aplicabilidade de acordo com uma configuração de sensibilidade pode ser ajustada para 33.4% para obtenção do melhor custo benefício, trazendo eficiência na assertividade do modelo.

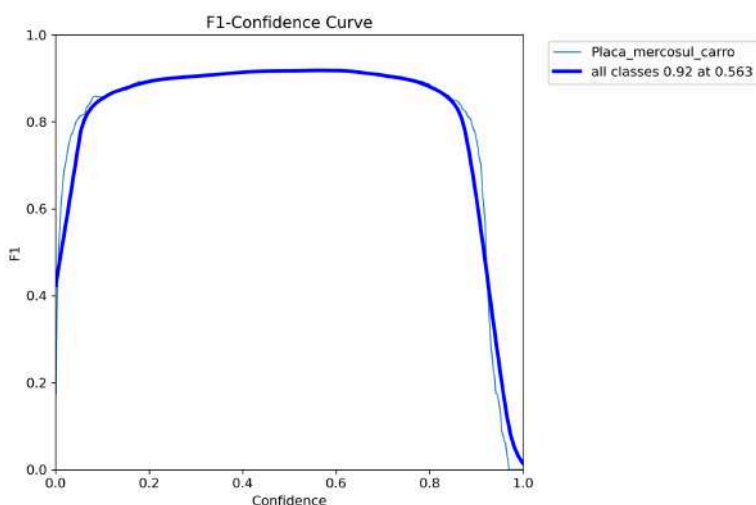


Figura 14 – F1-Score comparado com a confiança gerado pelo YOLO

A matriz de confusão ilustrada na figura 15 traz os números brutos obtidos durante o treinamento e suas correlações. Nela, vemos que o modelo teve 262 (Verdadeiros Positivos) Possuindo uma **acurácia de 91%** e **precisão de 88%**. Falhou cerca de 25 vezes no momento de detectar uma placa. Também acabou por detectar 34 vezes (Falsos Positivos) placas em locais onde não haviam. É nulo a quantidade de verdadeiros negativos.

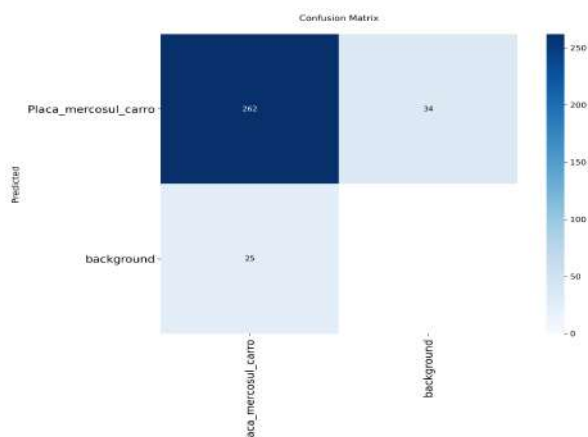


Figura 15 – Matriz de confusão gerada pelo YOLO.

4. Avaliação do Sistema a Partir dos Testes e Seu Resultados.

Foram realizados cerca de 50 testes de imagens de placas veiculares variadas divididas categoricamente em dois grupos contendo ocasionalmente imagens discrepantes. O **primeiro grupo** é composto de imagens de placas sóbrias, apresentando variações de tamanhos e cores e o **segundo grupo** de imagens placas retiradas em carros, apresentando cenários distintos com algumas possuindo variação angular acentuada.

4.1. Terminologias utilizadas na avaliação dos testes.

O meio de avaliação emprega três medidas: *Satisfatório*, *Medíocre*, *Insatisfatório*.

- I. **Satisfatório:** A placa foi detectada e a saída do programa correspondia ao valor exato da placa sem discrepância ou variação igual a somente um caractere.
- II. **Medíocre:** A placa foi detectada e a saída do programa corresponde ao valor da placa com discrepância de dois caracteres de diferentes tipos, seja por trocas cometidas pela OCR ou por causa da angulação da captura.
- III. **Insatisfatório.** A placa foi detectada porém não houve saída ou a discrepância foi superior ou igual a três caracteres de diferentes tipos, seja por trocas cometidas pela OCR ou por causa da angulação da captura.

4.2. Resultados dos testes na captura em tempo real

No conjunto de 50 imagens, o Tesseract foi capaz identificar todos os caracteres de 42 placas, uma taxa de 84%. Os casos de não identificação dos caracteres foram quase todos ocasionados pela angulação da placa. Das identificações que se demonstraram satisfatórias correspondem a 74% de todo o conjunto, já as saídas satisfatórias sem nenhuma discrepância entretanto, representaram 40% de todo o conjunto, estando quase majoritariamente no

primeiro grupo de imagens.

No conjunto, 6 resultados de testes foram considerados medíocres e 6 foram insatisfatórios. Existe uma correlação direta na forma de captura de imagem com a taxa de assertividade e precisão do sistema. Levando em conta os testes do primeiro grupo, percebe-se que em condições ideais o algoritmo possui uma precisão de 92%.

5. Limitações do Projeto e Propostas de Aperfeiçoamento

A OCR por ser uma biblioteca especializada em identificação de caracteres, nativamente, possui a capacidade de inferir acerca de uma gama variada de caracteres em formatos e tamanhos distintos. Todavia, uma ORC como o Tesseract apresenta dificuldades na leitura desses textos quando se apresentam de maneira diferente a qual o modelo foi feito para identificar (Texto horizontais).

Deste modo, a angulação da captura de certas imagens ocasiona uma leitura insatisfatória da placa ou medíocre. Contudo, muitas leituras bem sucedidas, certos caracteres demonstraram variações conforme os frames em suas próprias categorias. Apesar de ser possível corrigir através das expressões regulares o tipo de determinado caractere, convertendo letra para número e vice-versa, isso não evita erros dentro de uma mesma categoria.

As placas do modelo Mercosul que utilizam oficialmente a fonte FE-Engschrift e possuem no número “0” uma estreita risca. Essa risca apesar de estreita, mantém-se durante o processamento da imagem, e durante a interpretação, a OCR ao perceber uma espécie de “circunferência não fechada” Tende a inferir que é uma letra “o”, porém essa leitura não passaria nos testes de validação e seria corrigida rapidamente para um número equivalente. Entretanto, sendo uma característica da fonte FE-Engschrift possuir uma risca no zero, a OCR então tenta inferir algo próximo de uma circunferência não completa em tipos números, deste modo sugerindo então, o número “6”.

Para solucionar este desafio, poderia-se treinar a OCR em um processo onde ela gera uma série de arquivos de texto na fonte escolhida e a partir deles torna-se especialista no reconhecimento daquela fonte. Essa implementação deste modelo de OCR especializada aumentaria consideravelmente a quantidade de resultados satisfatórios sem discrepâncias.

CONSIDERAÇÕES FINAIS.

O presente trabalho apresentou o desenvolvimento e a avaliação de um sistema de Reconhecimento Automático de Placas Veiculares (ALPR) baseado em técnicas de Visão Computacional e aprendizado de máquina. A metodologia proposta estruturou-se em três etapas principais: detecção da placa por meio de uma Rede Neural Convolucional utilizando o modelo YOLOv8, processamento da imagem com auxílio da biblioteca OpenCV e extração dos caracteres através da OCR Tesseract, complementada por um processo de validação e correção utilizando expressões regulares.

Os resultados demonstraram que o modelo treinado apresentou desempenho satisfatório na etapa de detecção de placas, evidenciado pelas métricas obtidas durante o treinamento e pelos testes realizados com imagens e capturas em tempo real. O YOLOv8 mostrou-se eficiente na identificação das placas do padrão Mercosul, mesmo em cenários com variações de iluminação, ângulo e contexto visual.

Entretanto, observou-se que a etapa de reconhecimento de caracteres constitui o principal ponto de limitação do sistema. As dificuldades apresentadas pela OCR estão relacionadas principalmente à inclinação das placas e às características específicas da fonte utilizada no padrão Mercosul, fatores que ocasionam trocas entre caracteres visualmente semelhantes. Apesar dessas limitações, o uso de técnicas de pós-processamento com expressões regulares demonstrou ser uma estratégia eficaz para reduzir inconsistências e melhorar a confiabilidade das saídas.

Dessa forma, conclui-se que a arquitetura proposta apresenta potencial para aplicações práticas em sistemas de controle de acesso e monitoramento veicular. Como propostas futuras, recomenda-se o treinamento especializado da OCR para a fonte FE-Engschrift utilizada nas placas brasileiras, bem como a implementação de técnicas adicionais de correção de perspectiva e ampliação do dataset de treinamento. Tais aprimoramentos podem contribuir significativamente para o aumento da precisão do reconhecimento e para a consolidação do sistema como uma solução robusta no contexto de Sistemas Inteligentes de Transporte.

São Luís, Maranhão, 30 de abril de 2026



Documento assinado digitalmente

PEDRIANA DE JESUS PAVAO CASTRO

Data: 30/04/2026 12:06:50-0300

Verifique em <https://validar.iti.gov.br>

Assinatura do Orientador

REFERÊNCIAS

- BORGES, Vinicius Ruela Pereira. Processamento de Imagens: Quatro segmentações. São paulo, p1 - p6, 2013
- CONDOLETTE, Fabrício Lousada. SISTEMA PARA O GERENCIAMENTO E MONITORAMENTO DE TRÁFEGO DE VEÍCULOS, Espírito-Santo, p16 - 19, 2025.
- FERNANDES, Matheus Henrique. Identificação de placas de veículos utilizando reconhecimento de caracteres ópticos. Minas Gerais, p40 - p64, 2023.
- FIGUEIREDO, Lino Manuel Baptista. Sistemas Inteligentes de Transporte. p11-p20, Portugal, 2005.
- IBGE Brasil Pesquisas - Frota de Veículos. www.ibge.gov.br, 2024. Disponível em <https://cidades.ibge.gov.br/brasil/pesquisa/22/28120?tipo=grafico&indicador=28122>. Acesso em 30 de mar. de 2025
- LAROCA, Rayson. et al. On the cross-dataset generalization in license plate recognition. In: International Conference on Computer Vision Theory and Applications (VISAPP). Italia: SCITEPRESS, 2022. p. 168–177. ISBN 978-989-758-555-5. ISSN 2184-4321.
- MATAROZZI, Hugo Ricardo Ribeiro. Redes neurais e imagens: um estudo sobre redes convolucionais. São Paulo, p7 - p18, 2022.
- NETO, Aroldo de Sá Vargas. RECONHECIMENTO AUTOMÁTICO DE PLACAS VEICULARES DO MERCOSUL UTILIZANDO O TESSERACT OCR, Espírito Santo, p17 - p43, 2023.
- OLIVEIRA, G. S; SAMPAIO, G. S. Sistema de detecção de placas de carros brasileiras baseado em inteligência artificial, São Paulo, p2 - p17, 2024.
- OpenCV - Open Computer Vision Library. [Opencv.org](http://opencv.org). Disponível em: <https://opencv.org/releases/> Acesso em 29 de ago. 2025.
- PACHECO, Carlos Eduardo de Menezes. APLICAÇÃO DE VISÃO COMPUTACIONAL E TÉCNICAS DE RECONHECIMENTO DE CARACTERES EM PLACAS VEICULARES PARA CONTROLE DE ACESSO. Espírito Santo, p53 - p66, 2024.
- PÁTARRO, Miriam Gilberti. Mobilidade Urbana Sustentável: Desafios, inovações e políticas para cidades inclusivas. Questões Urbanas, São Paulo, v3, p3 - p7, 2024.
- RESENDE, P. T. V; SOUSA, P. R. MOBILIDADE URBANA NAS GRANDES CIDADES BRASILEIRAS: UM ESTUDO SOBRE OS IMPACTOS DO CONGESTIONAMENTO. Caderno de ideias, p5 - p25, Minas-Gerais, 2009
- Roboflow: Computer Vision Tools for developers and Enterprises www.roboflow.com. Disponível em <https://universe.roboflow.com/search?q=class%3Amercosul>. Acesso em 12 de

jul. de 2025.

- Roboflow - Placas-dec 3 (Classes) mercosul Computer Vision Dataset www.roboflow.com, 2025. Disponível em <https://universe.roboflow.com/teste-de-placas/placas-dec-3-classes-mercosul>. Acesso em 02 de fev. de 2026
- SCHLOSSER, T; FRIEDERICH, M et al A Consolidated Overview of Evaluation and Performance Metrics for Machine Learning and Computer Vision. Germany, p7 - p44, 2026
- SZELISKI, Richard. Computer Vision: Algorithms and Applications Second Edition. 2. ed. Estados Unidos: Springer Nature Switzerland AG, 2022.
- Documentação Ultralytics YOLO. ultralytics.com, 2023. Disponível em: <https://docs.ultralytics.com/pt/models/yolov8/#can-i-benchmark-yolov8-models-for-performance> Acesso em 5 de set. 2025.

ANEXOS

ANEXO A – Resumo publicado em Anais do Congresso Transdisciplinar CEST. São Luís, Maranhão, Brasil, 2025.



I CONGRESSO TRANSDISCIPLINAR CEST

**TRANSFORMANDO O MUNDO:
AGENDA 2030 E OS 17 OBJETIVOS
DE DESENVOLVIMENTO SUSTENTÁVEL (ODS)**

RESUMO SIMPLES

**RECONHECIMENTO DE PLACA DE CARRO COM APLICAÇÃO EM CONTROLE
DE ACESSO A ESTACIONAMENTO**

Samuel Pereira da Silva Varão¹
Pedriana de Jesus Pavão Costa²

RESUMO

No mundo moderno. O manejo inteligente de carros em diferentes tipos de espaços sempre foi um desafio. Seja em questões de logística, como a organização de distribuição de vagas, mobilidade no controle do acesso as entradas e saídas em estacionamentos, questões acerca da segurança e controle. Buscando ter uma maior abrangência de operação nesses ambientes, surge a alternativa. Desenvolver um sistema capaz de identificar as placas dos veículos. Essa identificação ajudaria a manter um controle logístico de forma a otimizar o tempo de espera em filas de acesso e saída, possibilitar uma distribuição organizada das vagas disponíveis e também aprimorar a segurança do local. Há diversas tecnologias desenvolvidas que podem ajudar na elaboração desse sistema. Baseadas nos princípios da visão computacional e no aprendizado de máquina, podemos utilizar inteligência artificial e redes neurais convolucionais. O Python e a biblioteca OpenCv unidos a algoritmos como o YOLO e Tesseract possibilitam a criação desse sistema. Desde o reconhecimento da placa veicular, manipulação e processamento da imagem obtida, leitura e reconhecimentos dos caracteres presentes na placa. Outros métodos como o Haas-cascade podem ser empregados no reconhecimento. O projeto tem como alicerce inicial o levantamento de uma base de imagens, advindas de bancos de imagens públicas e outras coletadas. O propósito a partir dessa base é criar um algoritmo que seja eficiente e confiável. Que possa ser capaz de fazer o reconhecimento das placas veiculares usando como alicerce a visão computacional e os recursos de processamento de imagem. Por tanto. Oferecer um sistema que também possa ser aprimorado, possibilitando um refinamento de seus processos, e futuramente executar o reconhecimento de outros elementos veiculares, como (cor, modelo, etc.) Além de uma possível integração a uma base de dados local para um maior controle e segurança do local.

Palavras-Chave: Visão computacional, Aprendizado de máquina, processamento de imagens.



Reitoria
Pró-Reitoria de Graduação
Coordenação de Pesquisa e Extensão
Assessoria de Pesquisa e Iniciação Científica
Programa Institucional de Iniciação Científica